

The Time Filter System: Advanced Time Manipulation for Animators and Effects Artists: How I Learned to Stop Worrying and Love Matrix Math

Brent Forrest*
brentforrest@duck.com
Marza Animation Planet Inc.
Tech Division
Japan

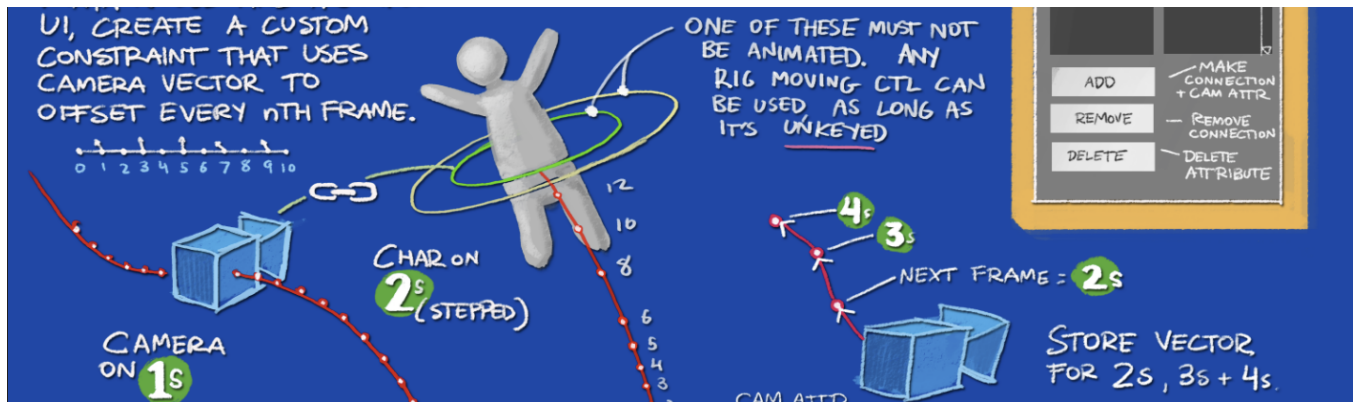


Figure 1: Time Filter original concept, 2019

ABSTRACT

Animation, in the classical sense, is all about timing and spacing. While many tools exist to aid in spatial manipulation, the Time Filter unlocks manipulation of time and offers new dimensions of control to the animation workflow.

CCS CONCEPTS

• Applied computing → Media arts.

KEYWORDS

animation, time, space, cg, stylized

ACM Reference Format:

Brent Forrest. 2023. The Time Filter System: Advanced Time Manipulation for Animators and Effects Artists: How I Learned to Stop Worrying and Love Matrix Math. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Talks (SIGGRAPH '23 Talks)*, August 06–10, 2023, Los Angeles, CA, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3587421.3595412>

*The author is the sole source of this research

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGGRAPH '23 Talks, August 06–10, 2023, Los Angeles, CA, USA

© 2023 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0143-6/23/08.

<https://doi.org/10.1145/3587421.3595412>

1 INTRODUCTION

One of the key challenges in achieving 2D appeal in CG is how to handle variable frame rates. Traditional "dropFrame" animation techniques, such as holding every other frame on "2s", are a crude oversimplification and can lead to jerky motion, slipping, and other undesirable effects. Our system offers a solution to this problem by providing a set of tools for filtering and modifying animation in Maya by manipulating how time is interpolated on sets of nodes. With these tools, animators can create custom animation effects and fine-tune their work to achieve the 2D-styled look they desire. The plugin is easy to use, pipeline-friendly, and non-destructive, making it a novel and valuable addition to the animator's toolkit.

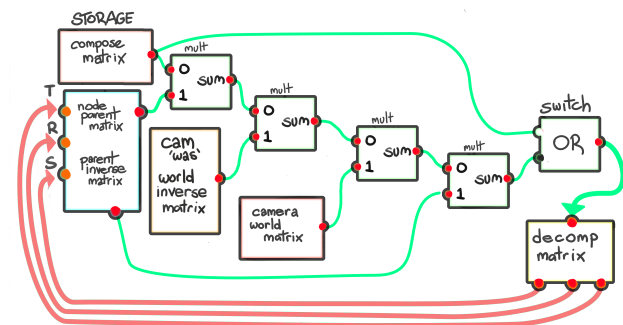


Figure 2: The Camera Offset module simplified.

2 DESIGN AND IMPLEMENTATION

The initial design came from a artist request about an existing tool which brute force bakes animation curves into stepped tangents, keyed on every other frame. The artist wished to add an option to skip frames where the character contacts the ground. It was clear that this was an opportunity to develop a system to eliminate the need to bake curves entirely, and allow the animator to toggle in and out of dropFrame mode at run time. The core of the problem was how to lock a Time-Filtered target to camera space during dropped frames. The CamOffset module in conjunction with the TimeFilter, constrains moving rigs to camera space on only frames specified by the data stream. The time-dependent offset formula was sorted out in the "Camera Offset Network" Module (Fig 2) and explained in further detail in the **advanced videos**. With OI as the offset local matrix and Pw as the as the parent world matrix and $Cw(t-1)$ as camera world matrix at a given time and $Cw(t)$ as camera world matrix at time t the formula is expressed as follows:

$$Pw(t+1) = OI(t) \times Pw(t) \times Cw(t-1) \times Cw(t) \times Pw(t-1)$$

3 UTILITY ARCHITECTURE

Time Filter is a collection of utilities and modules that modify animation timing. The base TimeFilter defines methods that can be implemented by other filters. These methods include controller appearance and behaviour, constructors which create nodes, and connection managers, which connect the attributes of the filter nodes to the control nodes. Additionally, the TimeFilter class manages the state of the filter, identifying when previously inanimate channels become animated, and handlers for objects which have no identifiable rigs - including simulations, caches, and any nodes which affected by time somewhere in the upstream graph.

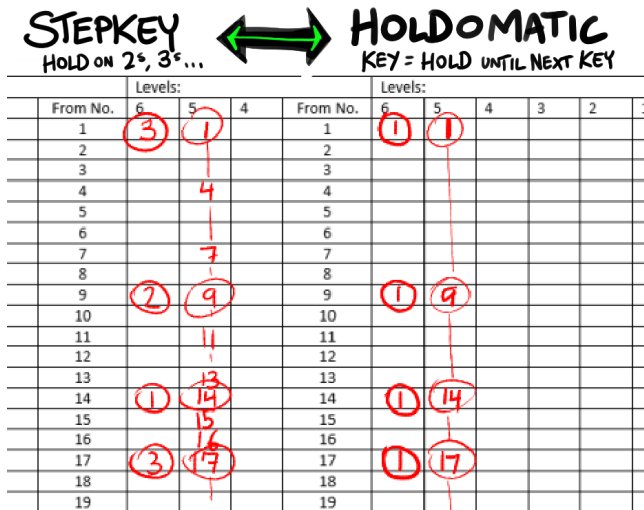


Figure 3: Dropframe versus Hold-O-Matic timing paradigm.

The DropFrameFilter class maintains a mapping between times and the amount of time that should be added or subtracted at that time. The node class uses this mapping to evaluate the filter and compute the adjusted time value. It also detects when an animation curve connected to the node's input attribute has changed, and updates the calculation accordingly.

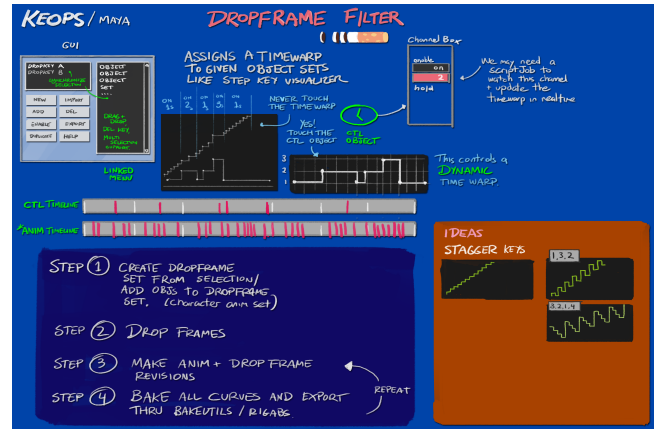


Figure 4: The initial DropFrame Filter proposal.

4 DISCOVERY

With the system in place, new concepts began to emerge. "Hold-O-Matic" (Fig 3), is a novel approach to dropframe timing developed in cooperation by the tech and animation teams. This method was notably preferred among CG animators who had no concept of traditional dope-sheet timing. Along with Drop-Frame we created Time Ramp and Time Reverse filters. Furthermore, large amount of research and development went into pipeline integration: reconstructing scenes that combined filters with constraints - for example: characters throwing objects while riding in vehicles, each with their own unique filters. These required additional pipeline modules: The 'IsInMulti' function for finding objects affected by multiple filters. 'GetConstrainedNodes' for identifying filtered objects that are constrained to unfiltered objects. Additionally, methods for finding animation curves hidden deep within the upstream graph. Together, these functions provide information used to modify filters or bake out animation, ensuring that baked animation contains only desired objects, and that filters are correctly re-applied after the baking process is complete.

5 CONTINUED EXPLORATION

One goal is to visualize time dilation on a per-vertex level. We also need to solve the problem of animated focal lengths. While it may seem like this only requires multiplying the view matrix, actually representing those results in the viewport is complex. This requires transforming targeted entities on a per-vertex basis (A challenge made more complex by the differential between animated nodes and the mesh they deform), or by building a novel viewport.

ACKNOWLEDGMENTS

This work would not have been possible without the technological guidance, encouragement and professional support of Gaetan Guidet, the animation expertise of Shashwat Brahmbhatt, the development assistance of Kim Sol and the leadership of Matsunari Takamasa. The author would also like to express thanks to the reviewers of this paper in 2019 and 2022 respectively, for their guidance on how to write a more acceptable ACM paper.